5. 실행 엔진 (Execution Engine) 분석

개요

Reth의 실행 엔진은 Ethereum 트랜잭션과 블록을 실행하는 핵심 컴포넌트입니다. 이 시스템은 revm(Rust Ethereum Virtual Machine)을 기반으로 구축되어 있으며, 모듈화된 설계를 통해 높은 성능과 유연성을 제공합니다.

실행 엔진의 주요 책임은 다음과 같습니다:

- 트랜잭션 실행 및 상태 업데이트
- 블록 검증 및 실행
- EVM 환경 구성 및 관리
- 가스 계산 및 수수료 처리
- 시스템 호출 및 하드포크 규칙 적용

핵심 구조체 및 트레이트

EthEvmConfig

`EthEvmConfig` 는 Ethereum EVM 설정을 담당하는 메인 구조체입니다.

```
#[derive(Debug, Clone)]
pub struct EthEvmConfig<C = ChainSpec, EvmFactory = EthEvmFactory> {
/// 내부 블록 실행기 팩토리
pub executor_factory: EthBlockExecutorFactory<RethReceiptBuilder, Arc<C>, EvmFa
/// Ethereum 블록 어셈블러
```

```
pub block_assembler: EthBlockAssembler<C>,
}
```

주요 메서드:

- `mainnet()` : 메인넷용 기본 설정 생성
- `new(chain_spec)` : 체인 스펙으로 새 설정 생성
- `evm_env(&header)` : 헤더 기반 EVM 환경 생성
- `next_evm_env(&parent, &attributes)` : 다음 블록용 EVM 환경 생성

ConfigureEvm 트레이트

EVM 설정을 위한 핵심 트레이트로, 실행 환경의 모든 측면을 정의합니다.

```
pub trait ConfigureEvm: Clone + Debug + Send + Sync + Unpin {
type Primitives: NodePrimitives;
type Error: Error + Send + Sync + 'static;
type NextBlockEnvCtx: Debug + Clone;
type BlockExecutorFactory: BlockExecutorFactory;
type BlockAssembler: BlockAssembler;


// 핵심 메서드들
fn evm_env(&self, header: &Header) -> Result<EvmEnv, Self::Error>;
fn next_evm_env(&self, parent: &Header, attributes: &Self::NextBlockEnvCtx)
-> Result<EvmEnv, Self::Error>;
fn executor<DB: Database>(&self, db: DB) -> impl Executor<DB>;
```

```
}
```

BlockExecutor와 Executor

블록 실행을 담당하는 핵심 트레이트들입니다.

```
pub trait Executor<DB: Database>: Sized {
type Primitives: NodePrimitives;
type Error;

/// 단일 블록 실행
fn execute_one(&mut self, block: &RecoveredBlock)
-> Result<BlockExecutionResult, Self::Error>;
/// 배치 블록 실행
fn execute_batch<'a, I>(&mut self, blocks: I)
-> Result<ExecutionOutcome, Self::Error>;
/// 상태 훅과 함께 실행
fn execute_with_state_hook<F>(&mut self, block: &RecoveredBlock, hook: F)
-> Result<BlockExecutionOutput, Self::Error>;
}
```

주요 기능

1. EVM 환경 구성

EVM 환경은 블록 실행에 필요한 모든 컨텍스트를 포함합니다:

```
// 헤더 기반 EVM 환경 생성
let evm_env = evm_config.evm_env(&header)?;


// 환경 구성 요소
pub struct EvmEnv {
pub cfg_env: CfgEnv, // 체인 설정 (체인 ID, 스펙 등)
pub block_env: BlockEnv, // 블록 환경 (번호, 타임스탬프, 가스 한도 등)
}
```

설정 과정:

1. 체인 스펙에서 하드포크 규칙 결정
2. 블록 헤더에서 환경 변수 추출
3. EIP-4844 blob 파라미터 설정
4. 가스 한도 및 기본 수수료 계산

2. 트랜잭션 실행 과정

트랜잭션 실행은 다음 단계를 거칩니다:

```
// 1. 트랜잭션 환경 설정
let tx_env = evm_config.tx_env(&transaction);


// 2. EVM 인스턴스 생성
let mut evm = evm_config.evm_with_env(db, evm_env);
```

```
// 3. 트랜잭션 실행
let result = evm.transact()?;


// 4. 상태 변경 적용
evm.db_mut().commit(result.state);
```

실행 단계:

1. **사전 검증**: 논스, 잔액, 가스 한도 확인
2. **가스 계산**: 내재적 가스 비용 계산
3. **EVM 실행**: 바이트코드 실행 및 상태 변경
4. **사후 처리**: 영수증 생성, 로그 수집

3. 블록 실행 및 검증

블록 실행은 모든 트랜잭션을 순차적으로 처리합니다:

```
impl<F, DB> Executor<DB> for BasicBlockExecutor<F, DB> {
fn execute_one(&mut self, block: &RecoveredBlock)
-> Result<BlockExecutionResult, Self::Error> {
// 블록 실행기 생성
let mut executor = self.strategy_factory
.executor_for_block(&mut self.db, block)?;
// 모든 트랜잭션 실행
let result = executor.execute_block(block.transactions_recovered())?;
```

```
// 상태 변경 병합
self.db.merge_transitions(BundleRetention::Reverts);
Ok(result)
}
}
```

4. 상태 업데이트 메커니즘

상태 업데이트는 계층적 구조로 관리됩니다:

```
// 상태 변경 추적
pub struct BundleState {
pub state: HashMap<Address, BundleAccount>,
pub contracts: HashMap<B256, Bytecode>,
pub reverts: HashMap<u64, HashMap<Address, RevertToSlot>>,
}

// 상태 변경 적용
db.merge_transitions(BundleRetention::Reverts);
let state_root = state_provider.state_root_with_updates(hashed_state)?;
```

revm 통합 방식

1. 데이터베이스 어댑터

Reth는 `StateProviderDatabase` 를 통해 revm과 통합됩니다:

```
#[derive(Clone)]
pub struct StateProviderDatabase<DB>(pub DB);

impl<DB: EvmStateProvider> Database for StateProviderDatabase<DB> {
type Error = ProviderError;

fn basic(&mut self, address: Address) -> Result<Option<AccountInfo>, Self::Err
Ok(self.basic_account(&address)?.map(Into::into))
}

fn code_by_hash(&mut self, code_hash: B256) -> Result<Bytecode, Self::Error> {
Ok(self.bytecode_by_hash(&code_hash)?.unwrap_or_default().0)
}

fn storage(&mut self, address: Address, index: U256) -> Result<U256, Self::Err
Ok(self.0.storage(address, B256::new(index.to_be_bytes()))?.unwrap_or_default())
}
}
```

2. 트랜잭션 환경 변환

Reth 트랜잭션을 revm 환경으로 변환:

```
impl TransactionEnv for TxEnv {
fn set_gas_limit(&mut self, gas_limit: u64) {
self.gas_limit = gas_limit;
}

fn set_nonce(&mut self, nonce: u64) {
self.nonce = nonce;
}

fn set_access_list(&mut self, access_list: AccessList) {
self.access_list = access_list;
if self.tx_type == LEGACY_TX_TYPE_ID {
self.tx_type = EIP2930_TX_TYPE_ID;
}
}
}
```

3. 검사기(Inspector) 시스템

revm의 검사기 시스템을 활용한 고급 기능:

```
// 상태 훅과 함께 EVM 생성
let evm = evm_config.evm_with_env_and_inspector(db, evm_env, inspector);
```

```
// 실행 중 상태 모니터링
fn execute_with_state_hook<F>(&mut self, block: &RecoveredBlock, state_hook: F
where F: OnStateHook + 'static {
let result = self.executor
.with_state_hook(Some(Box::new(state_hook)))
.execute_block(block.transactions_recovered())?;
}
```

최적화 기법

1. 상태 캐싱

```
// 캐시된 데이터베이스 사용
let cached_db = CacheDB::new(state_provider);
let mut evm = evm_config.evm_with_env(cached_db, evm_env);
```

2. 배치 실행

```
// 여러 블록을 배치로 실행
fn execute_batch<'a, I>(&mut self, blocks: I) -> Result<ExecutionOutcome, Self
let mut results = Vec::new();
for block in blocks {
results.push(self.execute_one(block)?);
```

```
}
Ok(ExecutionOutcome::from_blocks(first_block, self.into_state().take_bundle(), r
}
```

3. 메모리 관리

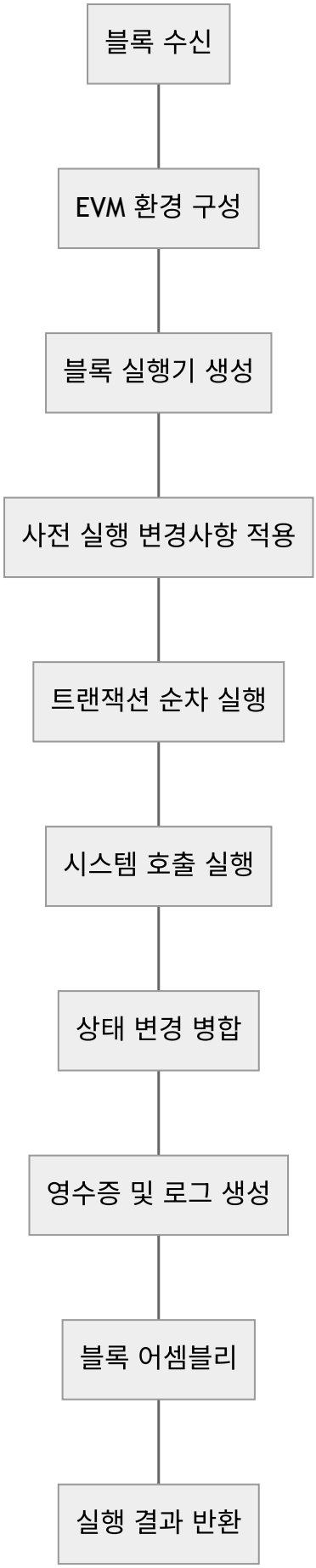
```
// 상태 변경 보존 정책
db.merge_transitions(BundleRetention::Reverts);

// 크기 힌트 제공
fn size_hint(&self) -> usize {
self.db.bundle_state.size_hint()
}
```

4. 병렬 처리 준비

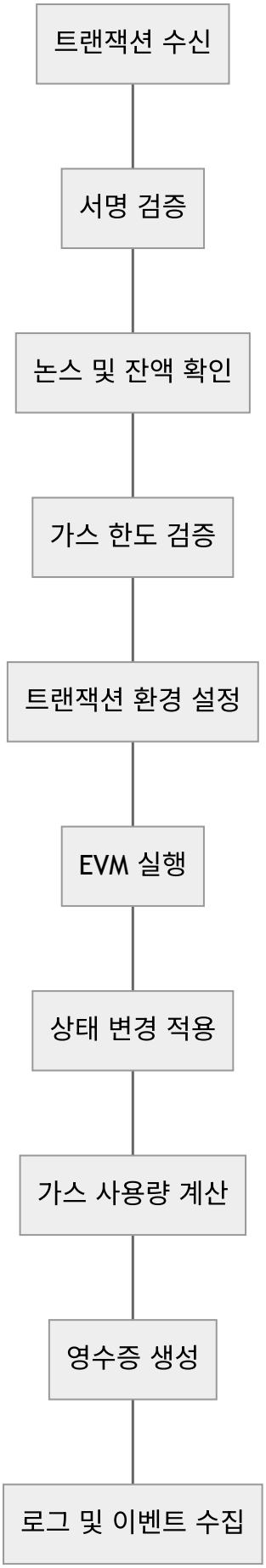
```
// Send + Sync 트레이트 구현으로 병렬 처리 지원
pub trait ConfigureEvm: Clone + Debug + Send + Sync + Unpin {
// ...
}
```

프로세스 플로우

블록 실행 플로우


블록 수신
EVM 환경 구성
블록 실행기 생성
사전 실행 변경사항 적용
트랜잭션 순차 실행
시스템 호출 실행
상태 변경 병합
영수증 및 로그 생성
블록 어셈블리
실행 결과 반환

트랜잭션 실행 플로우


트랜잭션 수신
서명 검증
논스 및 잔액 확인
가스 한도 검증
트랜잭션 환경 설정
EVM 실행
상태 변경 적용
가스 사용량 계산
영수증 생성
로그 및 이벤트 수집

하드포크 지원

동적 스펙 결정

```
// 타임스탬프와 블록 번호 기반 스펙 결정
let spec_id = revm_spec_by_timestamp_and_block_number(
chain_spec,
attributes.timestamp,
parent.number() + 1,
);

// 하드포크별 설정 적용
if self.chain_spec().is_osaka_active_at_timestamp(timestamp) {
cfg.tx_gas_limit_cap = Some(MAX_TX_GAS_LIMIT_OSAKA);
}
```

EIP 구현 예시

EIP-4788 (Beacon Block Root):

```
// 비콘 블록 루트 시스템 호출
if let Some(parent_beacon_block_root) = header.parent_beacon_block_root {
let system_call = SystemCall {
caller: SYSTEM_ADDRESS,
to: BEACON_ROOTS_ADDRESS,
data: parent_beacon_block_root.as_slice(),
```

```
};
executor.execute_system_call(system_call)?;
}
```

EIP-4844 (Blob Transactions):

```
// Blob 가스 계산
if self.chain_spec().is_cancun_active_at_timestamp(timestamp) {
let blob_gas_used = transactions.iter()
.map(|tx| tx.blob_gas_used().unwrap_or_default())
.sum();
let blob_gasprice = blob_params.calc_blob_fee(excess_blob_gas);
}
```

오류 처리

실행 오류 계층

```
#[derive(Debug, thiserror::Error)]
pub enum BlockExecutionError {
#[error("Validation error: {0}")]
Validation(#[from] BlockValidationError),
#[error("Internal execution error: {0}")]
Internal(#[from] InternalBlockExecutionError),
```

```
#[error("Provider error: {0}")]
Provider(#[from] ProviderError),
}
```

복구 가능한 오류 처리

```
// 트랜잭션 실행 중 오류 처리
match  executor.execute_transaction(tx) {
Ok(gas_used) => {
// 성공적인 실행
total_gas_used += gas_used;
}
Err(BlockExecutionError::Validation(err)) => {
// 검증 오류 - 트랜잭션 건너뛰기
warn!("Transaction validation failed: {}", err);
continue;
}
Err(err) => {
// 치명적 오류 - 블록 실행 중단
return Err(err);
}
}
```

성능 특성

메모리 사용량

- **상태 캐시**: 자주 접근하는 계정 정보 캐싱
- **번들 상태**: 변경사항을 메모리에 누적 후 일괄 적용
- **가비지 컬렉션**: 불필요한 상태 정보 정리

실행 속도

- **JIT 컴파일**: revm의 최적화된 바이트코드 실행
- **사전 컴파일**: 자주 사용되는 컨트랙트 사전 컴파일
- **병렬 검증**: 서명 검증 등 병렬 처리 가능한 작업 분리

확장성

- **모듈화**: 각 컴포넌트의 독립적 교체 가능
- **플러그인**: 커스텀 검사기 및 훅 지원
- **체인별 최적화**: 체인 스펙에 따른 최적화 적용

이 실행 엔진 설계를 통해 Reth는 높은 성능과 유연성을 제공하면서도 Ethereum 프로토콜의 모든 요구사항을 충족합니다.